

Desarrollo Web con JavaScript

1. Introducción y Herramientas de Desarrollo

El desarrollo moderno en JavaScript requiere no solo el conocimiento del lenguaje, sino también el dominio de las herramientas que optimizan la productividad y permiten la depuración eficiente del código. JavaScript es el lenguaje de programación por excelencia de la web, pudiendo funcionar tanto en el cliente (navegadores web) como en los servidores (usando Node.js).

Entornos de Desarrollo (IDE) y Extensiones

Un Entorno de Desarrollo Integrado (IDE) como **Visual Studio Code (VSCode)** es esencial. VSCode es un editor gratuito y extensible desarrollado por Microsoft, compatible con Windows, macOS y Linux. Su potencia radica en las extensiones que permiten personalizar el flujo de trabajo:

- **ESLint:** Analiza estáticamente el código para encontrar errores o patrones problemáticos antes de la ejecución.
- **Prettier:** Un formateador de código que garantiza un estilo consistente de forma automática.
- **Live Server:** Proporciona un servidor de desarrollo local con recarga automática, ideal para visualizar cambios en tiempo real al manipular el DOM.
- **JavaScript (ES6) code snippets:** Ofrece fragmentos de código abreviados para acelerar la escritura en la sintaxis moderna de JavaScript.

Herramientas del Desarrollador del Navegador (DevTools)

Las **DevTools** son fundamentales para depurar JavaScript directamente en el entorno donde se ejecuta, y se accede a las mismas presionando **F12** en el navegador web. El panel **Sources** permite inspeccionar los archivos fuente, mientras que el panel **Console** es vital para ejecutar comandos rápidamente y ver mensajes de error. Una técnica de depuración superior al `console.log()` es el uso de **puntos de interrupción (breakpoints)**, que permiten pausar la ejecución en una línea específica para examinar el estado de todas las variables en ese instante. Las DevTools también ofrecen la pestaña **Watch** para supervisar expresiones específicas y la pestaña **Scope** para ver qué variables están definidas en el contexto actual.

2. Fundamentos de JavaScript

JavaScript es un lenguaje de tipado dinámico y multi-paradigma, lo que le otorga una gran flexibilidad para resolver problemas de diversas maneras.

Variables y Ámbitos (**var**, **let**, **const**)

La declaración de variables ha evolucionado para ofrecer mayor seguridad:

- **var**: La forma antigua de declarar variables. Tiene un ámbito global o de función, lo que puede causar errores al permitir la re-declaración de la misma variable sin previo aviso.
- **let**: Introducida en ES6, permite declarar variables con **ámbito de bloque (scope)**. Esto significa que una variable declarada dentro de un **if** o un **for** solo existe dentro de esas llaves, evitando conflictos con el ámbito global.
- **const**: Se utiliza para valores que no serán reasignados. Al igual que **let**, tiene ámbito de bloque, pero es de solo lectura tras su asignación inicial. Es una buena práctica usar **const** por defecto a menos que se sepa que el valor debe cambiar.

Tipos de Datos y Operadores

JavaScript maneja diversos tipos de datos, siendo los más comunes:

1. **String**: Cadenas de texto encerradas en comillas simples, dobles o invertidas (backticks).
2. **Number**: Valores numéricos, incluyendo enteros y decimales.
3. **Boolean**: Valores lógicos, **true** o **false**.
4. **Array**: Listas de elementos indexados que comienzan en la posición cero.
5. **Object**: Colecciones de propiedades (clave-valor) que representan entidades del mundo real.

Los **operadores aritméticos** permiten realizar cálculos matemáticos (+, -, *, /, %). Los **operadores relacionales** (==, ===, !=, !==, >, <) comparan valores; se recomienda el uso del **triple igual (===)** para comparaciones estrictas que validan tanto el valor como el tipo de dato. Los **operadores lógicos** (&&, ||, !) combinan condiciones booleanas.

Funciones y Hoisting

Las funciones son bloques de código reutilizables. Existen varias formas de definirlas:

- **Declarativas**: `function saludar() { ... }`. Se benefician del **hoisting**, un proceso donde JavaScript asigna las declaraciones en memoria durante la compilación, permitiendo llamarlas antes de su línea de definición en el código.
- **Expresadas**: Se asignan a una variable (`const saludar = function() { ... }`). Estas no tienen hoisting y deben definirse antes de usarse.

- **Funciones de Flecha (Arrow Functions):** Sintaxis compacta introducida en ES6 (`const saludar = () => { ... }`). Son ideales para callbacks y mantienen el contexto de `this` de forma distinta a las funciones tradicionales.

Estructuras de Control de Flujo

Permiten modificar la ejecución lineal del programa:

- **if...else:** Evalúa una condición y ejecuta un bloque si es verdadera y otro si es falsa.
- **switch:** Útil para múltiples condiciones basadas en un solo valor, como menús de opciones.
- **Bucles (for, while):** El bucle `for` es ideal para recorrer arrays mediante un índice. El bucle `while` se ejecuta mientras una condición sea verdadera. El moderno `for...of` permite iterar directamente sobre los elementos de un objeto iterable como un array de forma más legible.

3. Manipulación del DOM (Document Object Model)

El DOM es la interfaz de programación para los documentos HTML. Representa la página como un árbol de nodos que JavaScript puede modificar para crear sitios dinámicos.

Selección de Elementos

Para interactuar con el DOM, primero debemos seleccionar los elementos:

- `document.getElementById('id')`: Selecciona un elemento único por su atributo ID.
- `document.querySelector('.clase')`: Método versátil que utiliza selectores CSS para obtener el primer elemento coincidente.
- `document.querySelectorAll('.clase')`: Devuelve una lista de todos los elementos que coinciden con el selector.

Modificación de Elementos

Una vez seleccionado, podemos alterar sus propiedades:

- **Contenido:** `textContent` se usa para cambiar el texto plano de un elemento.
- **Estilos:** La propiedad `style` permite modificar el CSS directamente (por ejemplo `elemento.style.color = 'red'`).
- **Atributos:** `setAttribute('src', 'url')` cambia atributos específicos como la fuente de una imagen.

- **Clases:** `classList.add()`, `classList.remove()` y `classList.toggle()` gestionan las clases CSS del elemento de forma dinámica.

Manejo de Eventos

Los eventos son acciones del usuario que JavaScript puede detectar mediante `addEventListener`.

- **click:** Se activa al pulsar un elemento, como un botón para agregar productos a un carrito.
- **input:** Útil para detectar cambios en tiempo real dentro de un campo de texto.
- **submit:** Se activa al enviar un formulario. Es crucial usar `event.preventDefault()` para evitar que la página se recargue y permitir el procesamiento de datos con JavaScript.
- **Delegación de eventos:** En lugar de asignar un escucha a cada elemento hijo (lo cual es ineficiente para contenido dinámico), se asigna un único escucha al contenedor padre y se identifica el origen del clic mediante `event.target`.

4. JSON: Estructura y Manipulación

JSON (JavaScript Object Notation) es el estándar para el intercambio de datos en la web debido a su ligereza y facilidad de lectura tanto para humanos como para las computadoras.

Estructura de JSON

Aunque se basa en la sintaxis de objetos de JavaScript, JSON tiene reglas estrictas:

- Las claves deben estar siempre encerradas en **comillas dobles**.
- No admite funciones, símbolos ni valores `undefined`.
- Los valores permitidos son strings, números, arrays, objetos, booleanos y `null`.

Métodos de Manipulación

JavaScript ofrece métodos nativos para trabajar con este formato:

- **JSON.stringify(objeto):** Convierte un objeto de JavaScript en una cadena de texto en formato JSON. Es indispensable para almacenar objetos complejos en el **localStorage**, que solo admite strings.
- **JSON.parse(stringJSON):** Realiza el proceso inverso, convirtiendo una cadena JSON nuevamente en un objeto o array de JavaScript para su manipulación.

5. Programación Asíncrona

JavaScript es un proceso (dentro del navegador web) de un solo hilo, lo que significa que solo procesa una instrucción a la vez. Para no bloquear la interfaz durante tareas largas (como peticiones a servidores), utiliza la asincronía.

Callbacks

Un callback es una función que se pasa como argumento a otra y se ejecuta cuando una operación termina. El uso excesivo de callbacks anidados puede llevar al denominado "**Callback Hell**", que significa que tenemos un código difícil de leer y de mantener.

Promises (Promesas)

Representan el resultado eventual (éxito o fracaso) de una operación asíncrona. Tienen estados: *pending* (pendiente), *fulfilled* (resuelta) y *rejected* (rechazada). Se consumen mediante `.then()` para el éxito y `.catch()` para errores.

Async / Await

Es una sintaxis moderna que hace que el código asíncrono se lea como síncrono.

- **async**: Antepuesta a una función, garantiza que siempre devuelva una promesa.
- **await**: Pausa la ejecución de la función hasta que la promesa se resuelva, sin bloquear el hilo principal del navegador. Para el manejo de errores en este flujo, se envuelven las llamadas en bloques **try...catch**.

6. AJAX y Fetch API

AJAX (Asynchronous JavaScript and XML) permite actualizar partes de una página web sin recargarla completamente, intercambiando datos con un servidor en segundo plano.

La Fetch API

Es la interfaz moderna que reemplaza al antiguo `XMLHttpRequest` para realizar peticiones HTTP. Una petición típica con `fetch` involucra dos pasos asíncronos:

1. Realizar la petición a una URL (por defecto usa el método **GET**).
2. Procesar la respuesta del servidor, generalmente convirtiéndola de un flujo de datos a un objeto JSON mediante el método `.json()`. Por ejemplo:

```
const obtenerDatos = async () => {  
  try {  
    const res = await fetch('https://api.ejemplo.com/datos');  
    if (!res.ok) throw new Error('Error en la red');  
    const data = await res.json();  
    console.log(data); // Visualización de los datos traídos  
  } catch (error) {  
    console.error('Hubo un fallo:', error);  
  }  
}
```

Consumo de APIs REST

Las APIs RESTful son servicios que exponen recursos a través de URLs únicas y utilizan métodos HTTP estándar. Al consumir estos datos, es común iterar sobre los resultados (usando `map` o `forEach`) y utilizar **Templates HTML** y **Fragments** para inyectar la información en el DOM de manera eficiente, evitando el reflujo innecesario de la página (reflow).